

Connector

The connector enables the DTI to retrieve real time data from the customer's systems. It handles authentication with these systems and converts the data into the standardized formats used on the DTI.

Related topics

Basic information

This document references `itemId` multiple times. This is the ID generally used to identify any entity for which an AASX file is being generated. For general purposes, this ID can be whatever string is more convenient to the customer. If the customer's systems already use a certain format of product ID, ERP ID or similar, these can be used. The final ID for the asset and other values will be computed from it automatically using the rules defined in the DTI.

Due to these derived values and how the value is generally used, there are some restrictions on this value:

- Uniqueness: The value must perfectly identify a single entity
- Length: The value must be at least 1 and at most 2048 characters.
 - 2048 is a theoretical upper limit, values which are too big may cause problems for certain browsers or other tools which might need to read the data
 - Prefer to stick to values of no more than 50 characters if possible
- The ID can only contain characters allowed on XML Schema 1.0 strings
 - In short, it should comply with the following regex: `^[\x09\x0A\x0D\x20-\u007F\uE000-\uFFFF\u00010000-\u0010FFFF]*$`
- The ID should be URL encodable

For more in depth details on why these restriction exist, see the [AAS Identifier](#) type and its uses thought the AAS architecture.

Specification

Health

HTTP Method: `GET`

Endpoint: <https://your-service/health>

Motivation

This endpoint allows the DTI check the status of the connector and customer systems, enabling detection of various kinds of issues. Anonymous access to this call is recommended.

Response

- `200 OK` in case the connector is operational.
- `5XX Server Error` if the connector itself is not healthy or if it is currently unable to access customer systems (due to a connection error for example).

In both cases, the contents of the response are not consumed by the DTI, only the status code.

Versions

HTTP Method: `GET`

Endpoint: <https://your-service/versions>

Returns the version of the connector specification implemented by the service. It may also return the version of the connector application itself.

Motivation

This endpoint allows the DTI to verify which version of the connector specification is supported by the service. It can also provide the current version of the deployed connector application for operational transparency and troubleshooting.

Response

200 OK with `VersionsResponse`

```
c#
public class VersionsResponse
{
    public ServiceVersion Spec { get; set; }

    public ServiceVersion? Application { get; set; }
}

public class ServiceVersion
{
    public string Version { get; set; }
}
```

`VersionsResponse.spec`

- Required field
- Contains the version of the connector specification implemented by the service
- Must follow [Semantic Versioning 2.0.0](#)

`VersionsResponse.application`

- Optional field
- Contains the version of the connector application itself
- May be omitted or set to `null` if this information is not available
- Must follow [Semantic Versioning 2.0.0](#) when provided

`ServiceVersion.version`

- Required field
- String representation of a semantic version

- Must follow the Semantic Versioning 2.0.0 format, e.g. `1.0.0` , `2.3.4` , or `1.2.0-beta.1`

Example response:

```
json
{
  "spec": {
    "version": "1.0.0"
  },
  "application": {
    "version": "2.3.4"
  }
}
```

In case of an unexpected error, the endpoint should return `500 Server Error` .

Hierarchy Levels

HTTP Method: `GET`

Endpoint: <https://your-service/product/hierarchy/levels>

Describes the general structure of a product/asset classification system by enumerating its hierarchy levels.

Motivation

The Neoception DTI provides flexible means for on demand asset administration shell (AAS) generation, able to cover the whole product/asset portfolio of large companies. Using mapping rules in the Back Office UI, a Content Admin can choose structure and content of the generated AASs. These rules need input to work on in order to efficiently reflect the complex requirements of the AAS generation for such a large product/asset portfolio. At the point in time these rules are evaluated, they work on a collection of key/value pairs, in their entirety also called "rule context". In some scenarios, the location of a product/asset within a hierarchical classification system is an extremely valuable contribution to this rule context. Each customer of the Neoception DTI is free to choose the classification system which supports a Content

Admin in the best way possible to create efficient and powerful mapping rules for the required data transformation. The DTI Core just needs to know how many levels the classification hierarchy consists of and how they are named.

Response

200 OK with `List<ProductHierarchyLevel>`

```
public class ProductHierarchyLevel
{
    public short Level { get; set; }
    public string? Name { get; set; }
}
```

c#

Example response:

```
[
  {
    "level": 1,
    "name": "Segments"
  },
  {
    "level": 2,
    "name": "Main groups"
  },
  {
    "level": 3,
    "name": "Groups"
  },
  {
    "level": 4,
    "name": "Classes"
  }
]
```

json

Hierarchies

HTTP Method: `GET`

Endpoint: <https://your-service/product/hierarchies>

Motivation

The tree with the full hierarchy where products can be placed is sometimes useful for certain operations.

This call is currently not in active use, but may be used on future updates. As such it may be implemented fully or left with a placeholder implementation.

Note: This endpoint must be available even if not fully implemented.

Response

`200 OK` with `List<ProductHierarchyNode>` as the body if fully implemented. `200 OK` with `[]` body if left as placeholder.

```
public class ProductHierarchyNode : ProductHierarchyLevel
{
    public List<ProductHierarchyNode> Children { get; set; } = new();
}
```

c#

json

```
[
  {
    "level": 1,
    "name": "27 Electric engineering, automation, process control
engineering",
    "children": [
      {
        "level": 2,
        "name": "27-06 Cable, wire",
        "children": [
          {
            "level": 3,
            "name": "27-06-01 Energy cable",
            "children": [
              {
                "level": 4,
                "name": "27-06-01-07 Energy cable >= 1 kV, for
permanent installation",
                "children": []
              }
            ]
          }
        ]
      },
      {
        "level": 2,
        "name": "27-28 Identification",
        "children": [
          {
            "level": 3,
            "name": "27-28-04 Inductive identification",
            "children": [
              {
                "level": 4,
                "name": "27-28-04-01 Identification, RFID Reader",
                "children": []
              }
            ]
          }
        ]
      }
    ]
  }
]
```

```

    ]
  },
  {
    "level": 1,
    "name": "28 Vehicle (complete vehicle)",
    "children": [
      {
        "level": 2,
        "name": "28-01 Automobile",
        "children": [
          {
            "level": 3,
            "name": "28-01-01 Passenger car",
            "children": [
              {
                "level": 4,
                "name": "28-01-01-07 Electric car (EV)",
                "children": []
              }
            ]
          }
        ]
      }
    ]
  }
],
{
  "level": 1,
  "name": "29 Home economics, Home technology",
  "children": [
    {
      "level": 2,
      "name": "29-15 Major domestic appliance",
      "children": [
        {
          "level": 3,
          "name": "29-15-01 Refrigerator, freezer (household)",
          "children": [
            {
              "level": 4,
              "name": "29-15-01-01 Refrigerator (household)",
              "children": []
            }
          ]
        }
      ]
    }
  ]
}

```

```

    ]
  },
  {
    "level": 3,
    "name": "29-15-03 Dishwasher, laundry appliance",
    "children": [
      {
        "level": 4,
        "name": "29-15-03-01 Dishwasher (household)",
        "children": []
      }
    ]
  }
]
}
]
}
]
}
]

```

Data Model

HTTP Method: **GET**

Endpoint: <https://your-service/model>

Enumerates all data source systems properties available for a Content Admin to map to the asset administration shell target structure properties.

Motivation

A unique list of property IDs is required to fetch specific data from the client's system related to a product. This endpoint returns a list of properties, where each property contains an ID that must be unique, a name that is optional but can provide additional description for the property, and a property type that indicates whether it is a document (such as an image, PDF, SVG, others) or a standard text property. This model also supports the Filters field, that allows adding a dictionary which enables the Admin to more precisely filter each Property in the back-office.

Response

200 OK with `List<Property>`

```
public enum PropertyType
{
    Property,
    Document
}
```

c#

```
public class Property
{
    public string Id { get; set; }

    public string? Name { get; set; }

    public string? Description { get; set; }

    public PropertyType Type { get; set; }

    public Dictionary<string, HashSet<string>>? Filters { get; set; }

    public List<string>? SemanticIds { get; set; } = new()
}
```

c#

Property.id

- Intended to serve in machine-to-machine communication only. Therefore, its readability for humans is secondary.
- Every property ID must be unique.
- A subset of these IDs will be included in the request content of the `GET /product/{itemId}/values` call, depending on the concrete `itemId` and the configured asset administration shell content.
- The actual `Property.id` seen on the UI of the DTI back-office may be slightly changed depending on if certain features are in use.
- This ID is case-sensitive
- There is no pre-established format for the product IDs

Property.name

- Intended to support a Content Admin to easily recognize the source system properties, therefore a good human readability is recommended
- Although not explicitly required, these names should also be unique for all properties included in the response content, otherwise a Content Admin might face duplicate entries in the UI
- The use of `Property.name` is optional but recommended, otherwise a Content Admin needs to deal with the usually rather cryptic content of `Property.id`
- In case the Connector represents more than a single data source system, the origin of each property may be indicated using e.g. a specific prefix like PIM*, ERP*, PLM_ etc.

Property.Description

- Intended to support a Content Admin to easily recognize the source system properties, therefore a good human readability is recommended
- Not in active use, but will appear when checking the details of properties on the backoffice UI
- Optional value. If present, should provide details on the nature of the property

Property.type

```
enum PropertyType {
    Property = 0,
    Document = 1,
}
```

ts

- Specifies the property value content which will be returned later in the response content of the call `GET product/{itemId}/values`
- `0` indicates that the value of a property will be a primitive and scalar like e.g. a single string, integer, float, double etc.

- **1** announces that the value of a property will be a document (such as an image, PDF, SVG, others). Choose this if either
 - the document content is directly returned as base64-encoded string value or
 - the property value is a URL referencing the document on a freely accessible server or
 - the document is e.g. contained on a restricted server and the value of this property is just an intermediate ID/voucher/ticket for this server which needs to be resolved in a second step after `GET product/{itemId}/values` using `POST /product/{itemId}/documents` to retrieve the actual document

Property.Filters

- Filters are optional key to value array objects that can be used for further filtering by the DTI.
- Each value should be unique within each list, but they can appear on multiple keys
- There are no predefined values or requirements by the DTI regarding these.
- They may be defined as necessary for the content admin.
- In general, they can be thought of as a way of adding custom values to the rule context.
- Filter keys are case-insensitive

Property.SemanticIds

- Intended for additional values that may be used as [semantic ids](#)
- Currently not in active use. However, will be used for automated mapping via matching semantic IDs from properties to the ones in submodels templates
- Values must obey the same general constraints as [AAS Identifiers](#)

Example response:

```
[
  {
    "id": "PIM_Thumbnail",
    "name": "PIM_Thumbnail",
    "description": "Thumbnail of the product",
    "type": 1
  },
  {
    "id": "PIM_EmailAddress",
    "name": "PIM_EmailAddress",
    "type": 0
  },
  {
    "id": "PIM_Website",
    "name": "PIM_Website",
    "type": 0
  },
  {
    "id": "PIM_ManufacturerProductDesignation",
    "name": "PIM_ManufacturerProductDesignation",
    "filters": {
      "source": ["PIM2"],
      "department": ["Department1", "Department2"]
    },
    "type": 0
  },
  {
    "id": "MarkingFile1",
    "name": "MarkingFile1",
    "type": 1,
    "semanticIds": ["0173-1#02-BAA120#008"]
  }
]
```

json

Product Hierarchy

HTTP Method: GET

Endpoint: <https://your-service/product/{itemId}/hierarchy>

This endpoint receives a `itemId` and returns the path to the location of the referenced product/asset within the hierarchical product/asset classification system as JSON in the response content.

Motivation

Rules can use Product Hierarchy filters and therefore, each time a request for a specific product is made we need to know where in the company hierarchy it is located to then know if a rule applies or not. For example instead of creating individual rules for 500 products in the "Groups" hierarchy, a content administrator can create a single rule that applies to all products at the "Groups" hierarchy level.

Response

200 OK with `List<ProductHierarchyLevel>` on success or **404 Not Found** if the provided `itemId` does not exist.

```
public class ProductHierarchyLevel
{
    public short Level { get; set; }

    public string? Name { get; set; }
}
```

c#

json

```
[
  {
    "level": 1,
    "name": "27 Electric engineering, automation, process control engineering"
  },
  {
    "level": 2,
    "name": "27-27 Sensor technology, safety-related sensor technology"
  },
  {
    "level": 3,
    "name": "27-27-09 Optoelectronic sensor"
  },
  {
    "level": 4,
    "name": "27-27-09-16 Optical distance sensor"
  }
]
```

Product Values

HTTP Method: **POST**

Endpoint(Post): <https://your-service/product/{itemId}/values>

This endpoint receives a `itemId` and the specification of the required property values for this product/asset as JSON in the content. In the context of this call, property values comprise text, numbers and documents. For textual property values like e.g. "left", "right", "low", "high" etc. and documents, language information is additionally provided. The call is expected to return the requested property values for the given asset as JSON in the response content.

Motivation

During the dynamic AAS generation, the DTI Core can create the target data structure with the means available in its own database. However, the intermediate result of these operations is just the skeleton of the target data structure, the still

missing parts are the concrete property values for the selected product/asset. In order to fill them in, the DTI Core asks the Connector for exactly the property values required to e.g. show a [submodel](#) in the DTI AAS Viewer or generate an AASX file. The request body contains a carefully selected subset of the `propertyId` values formerly retrieved using [GET /model](#), minimizing the query effort and the resulting data traffic volume. Depending on the quality of the data mapping configuration performed by the Content Admins, this call should only request property values available for the selected product/asset. In case of a mismatch in this regard, the Connector can simply omit the property values which are not available for a specific product/asset. The Connector is also free to return more property values than requested, e.g. for simplicity all property values available for a product/asset, although in most cases the performance of this call should be improved when making use of the given `propertyId` values in the request body to formulate more specific queries to the data source system.

Usage

This endpoint is integral part of the dynamic asset administration shell (AAS) generation. It will be triggered by a call to the official AAS API of the DTI Core, usage of the DTI AAS Viewer or full AASX file generation.

Request

This endpoint contains the following struct in the request body

c#

```

public class PropertyValuesRequest
{
    /// <summary>
    /// List of properties which value will change based on language
    /// </summary>
    public PropertiesWithLanguage PropertiesWithLanguage { get; set; }
    = new();

    /// <summary>
    /// List of properties which value won't change based on language
    /// </summary>
    public PropertiesWithoutLanguage PropertiesWithoutLanguage { get;
set; } = new();
}

public class PropertiesWithLanguage
{
    /// <summary>
    /// First item in the list represents the selected language in the
Viewer
    /// Second to last items in the list are the fallback languages
sorted by priority
    /// If empty, all languages available should be sent
    ///
    /// ISO 639 language codes should be used
    /// </summary>
    public List<string>? Languages { get; set; } = new();
    public List<string>? PropertyIds { get; set; } = new();
}

public class PropertiesWithoutLanguage
{
    public List<string>? PropertyIds { get; set; } = new();
}

```

PropertiesWithoutLanguage.propertyIds

Array of `propertyIds` with values which do not require the specification of languages. The contained `propertyIds` are a subset of what the Connector previously offered the DTI Core using the response content of the [GET /model](#) call.

The Connector should also accept `null` and an empty array `[]` as valid input.

PropertiesWithLanguage.propertyIds

Array of `propertyIds` with values that are expected to change depending on the specified language mixed with `propertyIds` representing documents (files). The contained `propertyIds` are a subset of what the Connector previously offered the DTI Core using the response content of the [GET /model](#) call. The Connector should also accept `null` and an empty array `[]` as valid input.

Natural Language Properties

Referenced properties should have textual property values like "low-active", "Class 2", "clocked" etc. The language dependency of a textual property is not decided by the data source system or the Connector. Instead, this depends on the associated [Submodel Element Type](#) (`MultiLanguageProperty`) in the target structure of the AAS currently being generated.

Document Properties

As of version 3.1.2, the [AAS meta model](#) doesn't explicitly support specifying if a document is language dependent or not. As a consequence, the DTI provides all document properties, no matter if their content value is in fact language dependent or not, in the `PropertiesWithLanguage.propertyIds` array and the Connector needs to step in to make this decision. If the Connector is able to determine that a document doesn't have any language dependency like e.g. a pictographic logo or a schematic drawing without any text, it may return a single document property value without language tag. Should the content of a document be language dependent, the same logic applies as for [Natural Language Properties](#).

PropertiesWithLanguage.languages

Array of language identification tags like specified by the [Internet Engineering Task Force in the Best Common Practice document 47](#). Generally speaking, it is not necessary to implement methods for parsing all code formats. However, 2-letter codes (e.g. "en", "de" and "pt") and language + region pairs (e.g. "en-US", "pt-PT") must be supported.

The Connector should also accept `null` and an empty array `[]` as valid input, in this case the Connector should query all existing property value + language tag combinations which are available in the data source system(s) and add all of them to the result array.

If this array in fact contains language tags, the Connector should try to query the property value matching the first language tag. On success, it should add this property value with its language tag to the result array and proceed to the next `propertyId`.

If the first language tag doesn't lead to success, the Connector should retry using the second and so on, until a single property value is found and added to the result array.

In other words, when concrete language tag values are provided there is need to retrieve more than one property value, the language tags can be interpreted as preferred language followed by fallback languages to show the user the best possible text value.

A typical use cases for this behavior is the visualization of an AAS in web-based Viewers where the user can select his preferred language in the browser.

When there are no more language tags left to try, either just skip this `propertyId` and do not add any data for it to the result array or fall back to e.g. the English ("en") language tag and add this result anyway.

Each Connector implementor should check with his business colleagues which options fit best to their needs.

Request example:

```
{
  "propertiesWithLanguage": {
    "languages": ["de", "en"],
    "propertyIds": [
      "PIM_Thumbnail",
      "PIM_Website",
      "PIM_ManufacturerProductDesignation",
      "MarkingFile1",
      "MarkingFile2"
    ]
  },
  "propertiesWithoutLanguage": {
    "propertyIds": []
  }
}
```

json

Response

200 OK with `List<ProductProperty>` .

```
public class ProductProperty
{
    public string PropertyId { get; set; }
    public string? Value { get; set; }
    public string? MimeType { get; set; }
    public string? FileName { get; set; }
    public ulong? Size { get; set; }
    public string? ValueLanguage { get; set; }
    public List<string>? Languages { get; set; }

    /// <summary>
    /// Whether a reference needs to be turned into the actual
    document by using GetDocuments
    /// </summary>
    public bool NeedsResolve { get; set; }
}
```

c#

ProductProperty.propertyId

References a `propertyId` given in `PropertiesWithoutLanguage.propertyIds` or `PropertiesWithLanguage.propertyIds`. In this response array, multiple `ProductProperty` objects may share the same `propertyId` value, as long as they differ in their `valueLanguage` value. This is the intended way of returning all available value + language tag tuples as reaction to an empty `PropertiesWithLanguage.languages` array.

ProductProperty.value

Holds the property value uniformly as string. Setting `value` to `null` is equivalent as not providing the `ProductProperty` object at all.

Textual Property Values

Textual property values can immediately be assigned to `value` without any conversion.

Other Data Types

Values of all other data types need to be converted to a string representation. The section [Enumeration for Submodel Element Value Types](#) of the AAS Meta model specification provides a good guideline for the implementation of this conversion. Especially "Table 3: Data Types with Examples" provides concrete examples for numerous data types - considering that the non-textual values should be seen here as strings as well, e.g. `"234.567e8"` for `xs:float` or `"true"` and `"false"` for `xs:boolean`.

Document Property Values

For properties that represent documents, the `Value` field may contain the Base64 encoded string of the value, a URL pointing to the resource or a ticker to be used for document redemption at a later point in time. See [Document Values Retrieval](#) for more details.

ProductProperty.valueLanguage

Holds the language tag correlating with the language of `value` if applicable. Refer to `PropertiesWithLanguage.languages` for the language tag specification. Should relate to a language tag provided in `PropertiesWithLanguage.languages`, if available. Only required for language dependent strings and documents. May be `null` or omitted for language independent values like numeric values, specific textual values (like e.g. "Android", "HTTPS" or "IEC 60529") and image file documents.

ProductProperty.mimeType

Specifies the document content type, if `value` holds a document. Should be absent or `null` for all other kind of data in `value`. A comprehensive list of media types can e.g. be found at the [Internet Assigned Numbers Authority](https://www.iana.org/assignments/media-types/).

ProductProperty.fileName

Only required for document properties, can be `null` or omitted for all other data types. When transforming a document into a file, it needs a name. The value provided in this property is the preferred name. To ensure maximum compatibility, the provided value should only contain a conservative set of characters, consisting of letters `A-Z` and `a-z`, numbers `0-9`, hyphen `-`, underscore `_` and period `.`. If used at all, the period character should only be present once to separate file name and extension. `fileName` does not need to specify a file extension, it is automatically derived from `mimeType` and appended. Should a file extension be provided, it will be overwritten by the file extension defined by `mimeType`.

If `fileName` is omitted or `null`, the fallback is a derivative of the [idShort](#) of the target AAS submodel element to which the document was assigned, supported by the value provided in `mimeType` for the file extension. Typical use cases for file creation are document downloads in the Neoception DTI AAS Viewer and the creation of an AASX file.

ProductProperty.size

Optional field. Can be used to communicate the size of documents not directly embedded on the response.

If not fully implemented or for properties which are not documents, should always contain `0`.

ProductProperty.languages

Optional field. Can be used to communicate all the languages for which a document is available. All these languages, including the one whose value is inside `valueLanguage` should be listed here.

If not fully implemented or for properties which are not documents, should always contain `[]`.

ProductProperty.needsResolve

Set to `true` for document values if their contents is to be retrieved following the **By Ticket With Subsequent Redemption** method. Omit or set to `false` in all other cases.

When `true`, the DTI will use the call `POST /product/{itemId}/documents` retrieve the actual document content.

The DTI will receive two separate object instances of type `ProductProperty`, one from the `POST /product/{itemId}/values` call and another from the `POST /product/{itemId}/documents` call. In both `ProductProperty` objects for this document, the common properties `valueLanguage`, `mimeType` and `fileName` should have the same value.

Response example:

json

```
[
  {
    "propertyId": "PIM_Thumbnail", // Document that already brings the
    content, needsResolve=False
    "value": "v8Y8XRaESWm4U1p...hh71CsYzYAAAAASUVORK5CYII=",
    "mimeType": "image/png",
    "filename": "284586-100008",
    "valueLanguage": "de",
    "languages": ["en", "de"],
    "needsResolve": false
  },
  {
    "propertyId": "PIM_Website", // The returned language is en
    because there was no value for de. You requested the properties for
    ["de", "en"].
    "value": "https://www.pepperl-fuchs.com/de-de/products-
    gp25581/107064",
    "valueLanguage": "en",
    "needsResolve": false
  },
  {
    "propertyId": "PIM_ManufacturerProductDesignation",
    "value": "Laserlichtschnitt-Sensor",
    "valueLanguage": "de",
    "needsResolve": false
  },
  {
    "propertyId": "MarkingFile1", // Document that the content-type is
    a URL
    "value": "http://name.pdf",
    "mimeType": "image/png",
    "filename": "MarkingFile1",
    "valueLanguage": "de",
    "languages": ["en", "de"],
    "needsResolve": false
  },
  {
    "propertyId": "MarkingFile2", // Document that brings a reference
    that needs to be redeemed. needsResolve=true
  }
]
```

```
"value": "DocRef0001",  
"mimeType": "application/pdf",  
"fileName": "MarkingFile2",  
"languages": ["en", "de"],  
"needsResolve": true  
}  
]
```

Product Documents

HTTP Method: **POST**

Endpoint(Post): <https://your-service/product/{itemId}/documents>

This endpoint is used to retrieve document data for **By Ticket With Subsequent Redemption** document retrieval. In this case it is the logical continuation after **POST** [/product/{itemId}/values](#) .

Motivation

Provide the option of fast and efficient UI rendering in the DTI AAS Viewer by delaying requesting the document content until it is in fact required.

Usage

The DTI AAS Viewer supports to just describe a document in a first step using the document meta data provided in the response content of **POST** [/product/{itemId}/values](#) . To keep track of the document, its property value is kept as a form of "ticket".

Request

This endpoint contains the following struct in the request body

c#

```

public class PropertiesWithLanguage
{
    /// <summary>
    /// First item in the list represents the selected language in the
    Viewer
    /// Second to last items in the list are the fallback languages
    sorted by priority
    /// If empty, all languages available should be sent
    /// </summary>
    public List<string>? Languages { get; set; } = new();
    public List<string>? PropertyIds { get; set; } = new();
}

```

json

```

{
  "languages": ["en"],
  "propertyIds": ["DocRef0001"]
}

```

PropertiesWithLanguage.propertyIds

Array of `propertyIds` with document "ticket" values.

PropertiesWithLanguage.languages

See the description of `PropertiesWithLanguage.languages` of the `POST /product/{assetId}/values` call.

Response

`200 OK` with `List<ProductProperty>`.

c#

```

public class ProductProperty
{
    public string PropertyId { get; set; }
    public string? Value { get; set; }
    public string? MimeType { get; set; }
    public string? FileName { get; set; }
    public ulong? Size { get; set; }
    public string? ValueLanguage { get; set; }
    public List<string>? Languages { get; set; }

    /// <summary>
    /// Whether a reference needs to be turned into the actual
    document by using GetDocuments
    /// </summary>
    public bool NeedsResolve { get; set; }
}

```

json

```

[
  {
    "propertyId": "DocRef0001",
    "value": "JVBERi0...KJHDH", // Content in base64
    "fileName": "DocRef0001.pdf",
    "needsResolve": false
  }
]

```

All item IDs

HTTP Method: **GET**

Endpoint(Post): <https://your-service/product/ids>

This optional endpoint is used to retrieve the **itemId** of all products provided by the connector.

Motivation

Some additional apps, most notably the Snapshot, have features that target every known item in the system. This call can be implemented to support these functions. If they are not desired, do not expose this endpoint.

Response

The response is a simple string array containing the gathered `itemId` s.

```
[  
  "REFRI001",  
  "CBL001",  
  "RFID02",  
  "EV009",  
  "CBL005",  
  "CBL009",  
  "REFRI004",  
  "RFID04",  
  "CBL002",  
  "RFID06"  
]
```

json

Document Value Retrieval

If a `ProductProperty.value` is intended to represent a document (file), Connector developers can choose between three methods to provide the document content.

Retrieval method	Allowed for images	<code>value</code> contents	<code>needsResolve</code>
By Reference	YES	Publicly accessible document URL	<code>false</code> or omitted
By Value	YES	Base64-encoded document content, supported only for image documents	<code>false</code> or omitted
By Ticket	NO	Internal ticket or identifier that can later be redeemed through <code>POST /product/{itemId}/documents</code>	<code>true</code>

Images Retrieval

As of the current DTI version, images must use either the `By Value` or `By Reference` retrieval method. This is scheduled to be relaxed in future versions of the DTI.

By Reference

This is the simplest option, `value` just holds a document URL pointing to the referenced document. The URL should be publicly and anonymously accessible, otherwise the option **By Ticket With Subsequent Redemption** is recommended.

Set `needsResolve` to `false` or omit it, the document content can be retrieved from another server, therefore no subsequent call to this Connector is required.

By Value

`value` directly holds the base64 encoded string representation of the document content. Set `needsResolve` to `false` or omit it, the document content is already present, therefore no subsequent call to this Connector is required.

By Ticket

In the method, the transmission of the actual document content is deferred. The `value` field just holds a preliminary "ticket" used for document retrieval.

Additionally, `needsResolve` must be set to `true` to identify that ticket retrieval is

The "ticket" value is an identifier string that is intended for internal use, it will not be part of the generated AAS. However, it should be enough for the `POST /product/{itemId}/documents` to be able to identify the exact document requested.

If at a later point in time the actual document content is required, this "ticket" will be provided to the call `POST /product/{itemId}/documents` as a `propertyIds` array element. Set `needsResolve` to `true` to inform the DTI accordingly.

This option can e.g. implement the scenario that a primary data source system holds URLs or other kind of identifying values pointing to a secondary, protected and internal document management system. The URL could become the "ticket" value for the subsequent call to `POST /product/{itemId}/documents`.

In less complex scenarios, the "ticket" value could simply be the `propertyId` of the document. This method also supports reducing data traffic in certain use cases by delaying the transmission of the document content to when it is actually required.

Connector Authentication

Connector to customer system authentication

Connectors can use whatever authentication method it is necessary to access the customer's systems and retrieve the data for requests. The implementation details vary according to the system being accessed and the requirements of the institution managing it. It is the job of the connector to abstract away such concerns from the perspective of the DTI core.

DTI to connector authentication

Connectors that require the DTI to authenticate to access them is a planned feature for the DTI 1.2.

The following authentication approaches are supported for DTI to connector communications:

- OAuth2 via client credentials
- Basic HTTP authentication

Users of such connectors must provide the required information for the DTI to access them. This either comes in the form of Username + Password for basic authentication or token endpoint/discovery endpoint, client ID, client secret and scope for OAuth2 authentication.